1. Contexte, besoin et périmètre

Audit du 12 novembre 2026 : livraison PARTIAL. Rotation confirmée par l'utilisateur ; évaluation humaine et vidéo non attestées. Les résultats guidés historiques ne constituent pas une mesure indépendante.

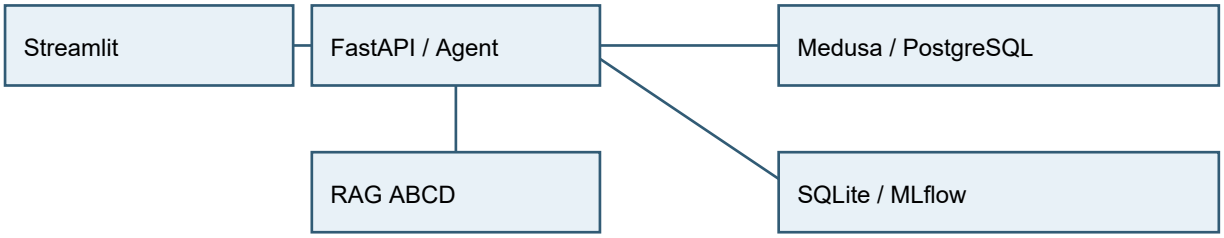
Le support e-commerce combine deux types d’information. Une documentation explique les procédures et les limites d’un service. Un système transactionnel contient l’état courant des produits, paniers et commandes. Confondre ces deux sources conduit à des réponses plausibles mais fausses : une procédure de remboursement ne démontre pas qu’un client est éligible, et un historique de commandes ne renseigne pas l’état d’une commande active.

CommerceOps AI vise un assistant capable de rechercher une référence, consulter une véritable boutique de démonstration et demander une confirmation avant une écriture sensible. Le prototype s’appuie sur le projet Python existant plutôt que sur une reconstruction. Les modules FastAPI, Streamlit, Medusa, RAG et SQLite ont été conservés et corrigés. Les actions autorisées sont la recherche de documentation et de produits, la consultation des commandes du client authentifié, la préparation d’un panier, la demande de checkout, la demande encadrée de retour et la création d’un ticket.

Le périmètre exclut les paiements réels, remboursements financiers autonomes, accès Admin depuis le LLM, données de comptes eBay et transformation des commandes historiques Olist en commandes actives. Un assistant ne doit pas déduire une autorisation de l’existence d’un article documentaire. Les décisions commerciales doivent être vérifiées par la boutique et les règles locales. Une demande en attente de confirmation ne représente pas une opération effectuée.

La réalisation a suivi P0 : installation, sources ABCD, API, sécurité, boutique réelle, preuve de checkout, observabilité et qualité. P1 comprend Chroma, Olist et le retour autorisé complet. P2 regroupe les extensions non nécessaires à la démonstration. Le parcours P0 fonctionne avec NVIDIA. Les extensions P1 exécutées ensuite sont Chroma, Olist et le retour complet. P2 reste différé conformément à la mission.

2. Architecture et responsabilités



L’interface Streamlit contacte FastAPI sur le port 8001. L’agent reçoit un message et expose des outils validés par Pydantic. Les outils documentaires consultent un index TF-IDF local. Les outils commerciaux utilisent uniquement la Store API de Medusa sur 9000 avec une clé publishable et, pour les opérations protégées, le JWT client. SQLite conserve les tickets, approbations et événements opérationnels. PostgreSQL contient les véritables données de boutique. MLflow stocke les expériences et traces expurgées.

Cette séparation permet de tester le code hors ligne avec des transports HTTP simulés tout en gardant une vérification d’intégration distincte sur un serveur réel. Les mocks ne valident pas les routes installées, les migrations, l’inventaire ou les paiements de test. Le script smoke_medusa exécute ces opérations contre Medusa 2.22.6 ; les assertions et résultats publics sont enregistrés séparément.

La configuration locale utilise PostgreSQL 16 et Redis 7 dans Docker, exposés uniquement sur la boucle locale aux ports 5433 et 6380. Le backend Medusa tourne sous Node 24.13.0 sur Windows. Son bus d’événements et verrouillage restent ceux par défaut, en mémoire. Redis est lancé mais les modules distribués ne sont pas configurés : cette installation est mono-instance et ne doit pas être décrite comme une infrastructure de production.

Le fournisseur choisi par l’utilisateur est NVIDIA. L’endpoint compatible OpenAI est https://integrate.api.nvidia.com/v1 ; le modèle configuré est nvidia/nemotron-3.5-lightning-30b-a3b. Le service refuse de produire une réponse si LLM_API_KEY est vide. Six scénarios ont effectivement appelé NVIDIA et les bons outils. Le modèle Llama initial a renvoyé HTTP 410 (fin de vie) ; Nemotron 3 Super était inaccessible (404). Nemotron 3.5 Lightning a ensuite fonctionné. Les résultats et IDs des runs sont dans reports/agent-live.json.

3. Environnement, installation et diagnostic

Le workspace contenait commerceops_ai avec les modules et neuf tests du starter. Aucun dépôt Git n’était initialisé. Le premier lancement de pytest a échoué faute de dépendance installée. Un environnement .venv a été créé sans modifier Python global. Le téléchargement initial a été refusé par la restriction réseau Windows ; la commande a ensuite réussi avec l’accès autorisé. Les versions exactes installées sont enregistrées dans requirements-windows-py314.lock.txt.

Python 3.14.3 était la seule version locale disponible. Les 42 tests ont été exécutés sur cette version, puis dans un conteneur Python 3.12 propre sans secrets et sur GitHub Actions. Le lock Windows reste spécifique à Python 3.14 ; celui du conteneur 3.12 est archivé séparément. Cette comparaison ne prétend pas à une reproduction bit à bit entre plateformes.

WSL était initialement inaccessible. Après démarrage de Docker Desktop, seule la distribution docker-desktop était visible. Le travail a donc continué sous PowerShell. Docker Engine 29.0.1 et Compose 2.40.3 ont permis de créer une nouvelle base et un nouveau volume dédiés au projet. Aucun volume utilisateur existant n’a été supprimé. Le starter Medusa a été cloné dans le dossier voisin my-medusa-store, SHA 0a84b2ac13dbf5091a84af4b54ce0b19f6c9ad55, puis installé avec pnpm 10.11.1 et son lockfile gelé.

Les migrations officielles ont réussi et semé une région Europe, le canal de vente, les options de livraison, les stocks, quatre produits et le fournisseur de paiement système de test. Les correctifs API ont été confrontés aux validateurs du paquet installé. Ainsi, customer_id ne doit pas être envoyé dans le JSON de création panier : Medusa le détermine à partir du JWT. Certains champs de commande nécessaires à l’autorisation doivent être demandés explicitement avec fields.

4. Sources, ingestion et provenance

ABCD provient du dépôt officiel ASAPP Research. La révision master observée est 6b8700ce67c6b37b062dd7a60abc76d7ef832a97, désormais utilisée par défaut pour les téléchargements reproductibles. Quatre fichiers ont été récupérés : guidelines.json (95 293 octets), kb.json (5 186), ontology.json (5 770), abcd_v1.1.json.gz (36 985 084). La limite initiale de 25 Mo rejetait les dialogues ; elle a été corrigée à 100 Mo pour permettre ce téléchargement réel.

Les consignes ont produit 55 documents, chacun identifié par le flux et sous-flux de recherche. Les dialogues sont conservés séparément et 100 premiers messages clients ont été sélectionnés de manière reproductible. Les familles d'outils attendues dans ce sous-ensemble sont des proxies heuristiques, pas des annotations humaines de réussite. Seules les consignes entrent dans le corpus indexé : aucun dialogue d'évaluation n'a été introduit dans les passages de recherche.

Un défaut de provenance a été corrigé : le code réutilisait la dernière URL de téléchargement, celle des dialogues, au moment d'enregistrer les consignes. Les documents pointent maintenant vers guidelines.json et le sous-flux correspondant. Les SHA256 des fichiers, paramètres et empreinte de l'index actif sont dans reports/source_manifest.json. Les fichiers bruts, corpus et index sont exclus de Git afin de maîtriser la redistribution et la taille du dépôt.

eBay n'a pas été collecté. L'accord explicite et la vérification des conditions restent nécessaires ; cette absence n'est ni un refus effectivement reçu du site ni une permission implicite. Olist a été téléchargé par l'endpoint public officiel Kaggle : HTTP 200 ZIP, neuf CSV. SQLite contient 99 441 commandes historiques ; le deuxième import n'a ajouté aucun doublon. Le prototype ne présente aucune de ces sources comme des commandes actives. ABCD contient des procédures de recherche parfois incompatibles avec les règles de cette boutique ; elles restent des données externes.

5. Recherche documentaire et expérimentation

Le moteur TF-IDF calcule la similarité cosinus entre une question et les passages. Il conserve titre, source, type et date de collecte. La réponse de recherche expose les extraits originaux ; aucune FAQ fictive n'a été ajoutée. Un vocabulaire explicite relie quelques termes français du support à leurs équivalents anglais, par exemple retour/return, couleur/color et commande/order. Ce pont lexical est limité : il ne remplace pas un modèle multilingue.

Un seuil de développement de 0,12 évite de retourner tous les passages ayant une intersection lexicale minime. Ce seuil n'est pas une probabilité de vérité ; il doit être calibré sur un jeu plus large revu humainement. Une question sans vocabulaire pertinent renvoie une liste vide. L'instruction système demande de reconnaître l'absence de source et de ne jamais traiter les consignes du corpus comme des instructions de l'assistant. Les tests vérifient la séparation entre texte documentaire et outils autorisés, mais ne prouvent pas une résistance générale du LLM aux injections.

Deux configurations ont été réellement comparées sur les mêmes douze requêtes de développement, dont quatre françaises. Les associations question/sous-flux ont été rédigées et inspectées par l'agent à partir des vraies consignes. Elles ne sont pas validées par un humain et n'offrent pas une estimation indépendante de généralisation. La métrique vérifie si une source attendue apparaît dans les cinq premiers passages ; elle ne mesure ni la fidélité d'une réponse générée ni la réussite d'une opération.

Configuration	Documents	Passages	Recall@5	Moyenne à chaud	P95 à chaud
90 mots, chevauchement 20	55	205	8/12 = 0,667	1,39 ms	2,07 ms
180 mots, chevauchement 35	55	120	6/12 = 0,500	1,13 ms	1,30 ms

Ces latences proviennent de l'exécution consignée dans reports/evaluation.json ; elles excluent le chargement/vectorisation initiale, l'API HTTP et le LLM. L'index actif utilise 90/20, sélectionné pour son meilleur rappel sur ce petit jeu. Deux runs MLflow référencés dans le rapport stockent paramètres, empreinte de l'index, métriques et prédictions. Chroma 1.5.9 et Sentence Transformers 5.7.0 ont ensuite été installés. L'index dense multilingue (même corpus de 205 passages) a obtenu 6/12, soit 0,500, en 23,75 ms de moyenne et 30,15 ms de P95 à chaud. Il a refusé la question inconnue. Son modèle est paraphrase-multilingual-MiniLM-L12-v2 à la révision e8f8c211226b894fcb81acc59f3b34ba3efd5f42. Le lexical demeure sélectionné pour son rappel supérieur. Le cache a dû être raccourci dans le dossier temporaire Windows après un échec MAX_PATH.

6. Outils, validation et consentement

L’agent expose huit outils : recherche documentaire, recherche produits, liste de commandes, suivi d’une commande, préparation panier, demande checkout, demande retour et création ticket. Les modèles Pydantic sont stricts : quantités entières bornées, identifiants validés, catégories limitées et champs supplémentaires refusés. Une chaîne numérique ou un booléen ne doit pas être accepté comme une quantité. Les erreurs de validation retournent un message expurgé pour éviter d’échoer un secret présent dans l’entrée invalide.

Les écritures commerciales passent par une approbation SQLite liée au client authentifié. La transition pending vers confirmed utilise une condition SQL atomique ; un double clic et huit confirmations concurrentes ne déclenchent qu’une seule prise en charge. L’appartenance est vérifiée avant la préparation et à nouveau lors de l’action effective. Les états done et failed reflètent l’issue observée ; les opérations échouées ne sont pas réessayées silencieusement afin d’éviter les doublons.

Les modèles de configuration ferment les retours par défaut ; ils ont été activés explicitement dans cette seule boutique de démonstration pour valider le parcours P1. Une recherche documentaire ne suffit plus à les autoriser. Si le marchand active les règles locales, le client Python vérifie l’état expédié, l’appartenance des articles et les quantités. Le middleware Medusa vérifie l’authentification, la propriété de la commande, interdit receive_now et exige une activation marchande distincte. Une demande réelle return_01M76G6DA7M4AJZ0J9YBNEA44Z a été créée après fulfillment de notre commande de test et confirmation API. La lecture Admin a confirmé sa persistance. La réception automatique et le remboursement financier n’ont pas été exécutés.

L’orchestrateur limite les itérations et le nombre total d’appels, puis s’arrête dès une demande de confirmation. Les tool_calls malformés sont refusés. Après une erreur d’outil, une réponse finale du modèle affirmant une réussite est remplacée par un message prudent d’échec et d’escalade. Cette défense ne remplace pas une évaluation réelle du modèle : la sélection d’outils a été vérifiée sur six scénarios guidés (6/6, un appel moyen). Les coûts et une mesure indépendante de fidélité restent non évalués.

7. Vérification de la boutique réelle

Le script d’intégration a créé deux comptes clients par les mécanismes d’authentification Medusa et obtenu leurs JWT sans les enregistrer dans les rapports. Les identifiants de connexion sont conservés uniquement dans un fichier local ignoré. Le catalogue renvoyait quatre produits. Une demande de préparation panier a créé une approbation réelle ; son propriétaire a pu la prendre une seule fois, tandis que le second client ne pouvait pas y accéder.

Un panier a ensuite été créé, enrichi d’un article, d’une adresse explicitement fictive de démonstration, d’une méthode de livraison et d’une session de paiement pp_system_default. La finalisation a effectivement produit la commande order_01M76EPFXQEJ8VPAC3N5TCSJ6R, statut pending. Cette commande existe dans PostgreSQL et est consultable via la Store API du client propriétaire. Le fournisseur de paiement système ne réalise aucun débit financier ; pending ne signifie pas livré.

Le client B a été refusé lors de la consultation de cette commande et lors du checkout du panier A. Un appel HTTP direct de retour sans JWT a reçu 401 ; le même appel avec le JWT B a reçu 403. Le checkout direct du panier A avec B a également reçu 403. Ces requêtes testent le vrai serveur et complètent les tests Python. Un ticket tic_244d866e86d24563 a été persisté dans SQLite et lié au propriétaire de la commande.

Le rapport reports/medusa-smoke.json contient les IDs techniques et statuts nécessaires à la vérification sans données d’authentification. Le script crée une nouvelle commande à chaque exécution : il doit être lancé intentionnellement et exclusivement sur la boutique locale de démonstration. Le storefront Next.js optionnel n’a pas été démarré ; le checkout a utilisé les API officielles. Cette première commande a été créée par le script API. Une deuxième commande order_01M76FXF3CQBMTZ81FGHD5N638 a ensuite été créée à partir d’un panier demandé par NVIDIA, configuré localement pour livraison/paiement de test, puis finalisé après demande LLM request_checkout et confirmation API. Un ticket a suivi. Cette deuxième commande a servi au retour P1.

8. Observabilité et qualité logicielle

Les audits SQLite enregistrent identifiant de requête, outil, catégorie opérationnelle, succès, durée, type d’erreur et horodatage. Les interactions ajoutent appels, itérations, modèle, confirmation et décision d’escalade. Les requêtes HTTP portent un X-Request-ID et une mesure de durée/statut liée à une route modèle, sans paramètres sensibles. L’endpoint /metrics calcule les erreurs, durées moyennes/P95 et taux de succès des appels observés. Ces statistiques décrivent les événements enregistrés, pas une qualité universelle de l’assistant.

Une trace applicative MLflow a été persistée et relue par son identifiant tr-796b5727ef370ae47927e6d054285d4c. Elle contient le span tool.search_knowledge et des attributs expurgés. La recherche correspondante avait found=false, ce qui est conservé honnêtement. Les traces n’enregistrent ni prompts, JWT, adresses ni réponses complètes. Cette réduction de contenu protège la confidentialité mais limite l’analyse de fidélité depuis la seule trace.

Les 42 tests hors ligne passent. Ils couvrent provenance, corpus vide ou absent, arguments invalides, isolation, concurrence, confirmation unique, statuts HTTP 401/403/404/409/429/500, timeout, réponses non conformes, limite totale d’outils et faux succès après erreur. Les tests de quality gate rejettent baseline absente, contrôles vides, métrique absente, valeur non finie et régression. Le lint de sécurité Python passe ; le lint officiel Medusa a aussi été exécuté.

Le workflow GitHub Actions installe les dépendances, lance lint et pytest, contrôle l'absence de fichiers sensibles et évalue le snapshot de benchmark contre la baseline. Ce snapshot ne remplace pas une nouvelle collecte et une nouvelle expérience. La baseline correspond au jeu de développement ; le script ne la remplace qu'avec une option explicite. Après autorisation de publication et contrôle des secrets, le commit 75ed8e7 a été publié et son run GitHub Actions 43143323117 a terminé avec succès.

9. Gouvernance, risques et limites

Les contrôles d’accès utilisent l’identité retournée par Medusa, jamais un customer_id choisi par le modèle. La clé publishable ne donne pas de privilèges Admin. Le mot de passe Admin généré localement et les comptes de démonstration sont exclus de Git. Le scanner de secrets ne révèle que les chemins des fichiers détectés ; il s’agit d’un contrôle simple, pas d’une garantie exhaustive d’absence de secret. Les erreurs réseau exposent une classe d’erreur sans corps distant ni en-têtes.

Les limites opérationnelles restantes sont documentées : absence de TTL d’approbation, quotas et politique de rétention, statut confirmed possible après crash, absence de coordination distribuée, validation limitée à un retour autorisé sur la boutique locale. Les remboursements financiers sont réservés à une décision humaine marchande. Le système n’expose aucun outil de transfert d’argent. Les textes ABCD contenant des procédures de recherche imprudentes ne peuvent modifier ces permissions.

La pertinence documentaire reste limitée : 8/12 sur un jeu rédigé par l’agent ne constitue pas une validation humaine. La généralisation en français, les questions ambiguës, la fidélité des citations et les attaques par documents doivent être évaluées avec un fournisseur réel et un jeu indépendant. Les coûts et tokens ne sont pas mesurés. La sélection d’outils est correcte sur 6/6 scénarios guidés, avec latence moyenne 4 285 ms ; cela ne mesure pas la généralisation. Deux scénarios négatifs supplémentaires ont confirmé le refus d’une commande étrangère et d’un remboursement financier. Leur absence est affichée au lieu d’être remplacée par des nombres arbitraires.

Les services écoutent localement. Une mise en ligne nécessiterait une revue supplémentaire : authentification applicative, restriction réseau, supervision marchande, sauvegardes, restauration, rétention, audit et gestion des secrets. L’état obtenu est un prototype académique vérifiable, pas un service client de production. eBay et Olist demeurent des sources prévues distinctes ; aucune permission de collecte n’est inventée pour compléter la démonstration.

10. Démonstration, reproduction et travaux restants

Les quatre services ont répondu HTTP 200 : API 8001, Streamlit 8501, Medusa 9000 et MLflow 5000. L’API distingue api=ok, medusa_reachable, index_available et llm_configured. À la première vérification sans clé, ce dernier était false ; après configuration NVIDIA, les quatre indicateurs étaient positifs. L’interface permet une recherche documentaire directe sans LLM, utile pour présenter les vraies sources et l’incertitude de récupération. Le chargement Streamlit a été testé sans exception avec AppTest.

Le scénario de démonstration montre d’abord la santé des composants, une recherche « retour mauvaise couleur », le catalogue et la commande de test, puis les refus 401/403, le ticket et les runs MLflow. La vidéo n’a pas été enregistrée et les captures restent à produire sans secrets. La documentation distingue le premier test API du parcours conversationnel NVIDIA ensuite exécuté et de ses limites. Les accès Admin locaux permettent de contrôler manuellement la commande, sans divulguer le mot de passe dans le rapport.

Les commandes de relance sont centralisées dans scripts/start.cmd : infra, medusa, api, ui, mlflow et check. La reconstruction des données, sources et expériences est détaillée dans REPRODUCIBILITY.md. Les versions, SHA, résultats et limites sont reliés aux fichiers reports/. Les correctifs serveur sont archivés dans integrations/medusa afin de pouvoir les réappliquer au clone officiel. Le workspace VS Code pointe vers la racine Python et son interpréteur isolé.

Le parcours réel couvre six outils, les confirmations et les refus. Chroma, Olist et le retour marchand local ont été exécutés. L'utilisateur a confirmé la révocation de la clé exposée et l'installation d'une nouvelle clé ; une lecture sûre du catalogue a réussi après relance de l'API. La publication et la CI sont vérifiées, et le PDF comporte 10 pages. Restent la revue humaine des annotations, l'évaluation indépendante complète, les captures et la vidéo réelle. Le parcours local fonctionne, mais la livraison globale reste PARTIAL ; eBay n'est pas collecté.

Références et preuves

- Mission fournie : CODEX_MISSION_COMMERCEOPS_AI.md, matrice des exigences du professeur ; PDF du cours absent du workspace. - ABCD officiel : https://github.com/asappresearch/abcd ; manifeste dans reports/source_manifest.json. - Medusa : https://github.com/medusajs/dtc-starter ; API installée 2.22.6 et ses AGENTS.md/validateurs inspectés. - Protection des retours : https://docs.medusajs.com/resources/commerce-modules/order/secure-return-creation . - NVIDIA : https://build.nvidia.com/nvidia/nemotron-3.5-lightning-30b-a3b/build . - Preuves locales : reports/evaluation.json, reports/medusa-smoke.json, reports/http-smoke.json, reports/pytest.xml, reports/trace-verification.json.

Compléments audit final : logs Python 3.14 et reproduction propre Python 3.12 dans reports/final-audit/. Jeu distinct gelé de 16 cas : revue humaine non attestée, mesures de récupération provisoires uniquement. Captures Admin/CI et vidéo non disponibles dans cette exportation.

Résultats du nouveau brouillon (même sélection gelée, n=5 questions avec source) : lexical 1/5, dense 2/5. Aucune réussite de tâche indépendante mesurée, n=0. Rotation de clé confirmée par l'utilisateur et lecture sûre réussie. Publication autorisée : SIMOS21/call_center-ai ; CI du commit 75ed8e7 réussie, https://github.com/SIMOS21/call_center-ai/actions/runs/43143323117 . La livraison globale reste PARTIAL, malgré les contrôles techniques réussis.